Inverse Probability Weighting and Marginal Structural Models

Davood Tofighi, Ph.D.

Lecture Notes: IP Weighting and Marginal Structural Models

Overview

Inverse Probability (IP) weighting and Marginal Structural Models (MSMs) represent a paradigm shift in causal inference from observational data. Unlike traditional regression approaches that model the outcome conditional on covariates, these methods focus on modeling the treatment assignment mechanism to create a pseudo-population where confounding is eliminated. This lecture provides a comprehensive exploration of these powerful techniques, their theoretical foundations, practical implementation, and nuanced considerations for applied researchers.

Learning Objectives

By the end of this session, students will be able to:

1. **Explain** the conceptual foundation and mathematical basis of IP weighting
2. **Differentiate** between stabilized and non-stabilized weights and their appropriate applications

3. **Implement** MSMs using IP weighting in R for various data scenarios
4. **Diagnose** and **address** common issues including positivity violations and model misspecification
5. **Interpret** MSM results and contrast them with traditional regression outputs
6. **Extend** these methods to complex scenarios including continuous treatments and time-varying exposures
7. **Communicate** results from MSM analyses to diverse audiences

R Packages

```r
pacman::p_load(
    # Core data manipulation and visualization
    tidyverse, # ggplot2, dplyr, tidyr, purrr, etc.
    data.table, # Efficient large data handling
    here, # File path management

    # Causal inference specialized packages
    dagitty, # DAG creation and analysis
    ggdag, # DAG visualization
    survey, # For weighted analyses and complex surveys
    MatchIt, # Propensity score matching
    WeightIt, # Advanced weighting methods
    tableone, # Balance tables
    cobalt, # Balance assessment and visualization
    marginaleffects, # Counterfactual predictions and
contrasts
    tmle, # Targeted maximum likelihood estimation

    # Statistical modeling
    sandwich, # Robust variance estimation
    lmtest, # Hypothesis testing for linear models
    boot, # Bootstrap methods

    # Visualization extras
```

```
    patchwork, # Combining ggplot2 plots
    ggridges, # Ridge plots for distribution
visualization
    scales # Better scale formatting
)
```

Comprehensive Topic Explanation

Theoretical Foundations of IP Weighting

Inverse Probability weighting addresses confounding by creating a pseudo-population where treatment assignment is independent of covariates (Hernán & Robins (2020)). The fundamental insight is that we can remove the association between covariates and treatment by reweighting observations according to their probability of receiving their actual treatment.

The IP weight for individual i is defined as:

$$W_i^A = \frac{1}{\Pr(A = a_i | L_i)}$$

where:

- A represents treatment assignment
- L represents observed covariates
- $\Pr(A = a_i | L_i)$ is the propensity score

Marginal Structural Models (MSMs)

MSMs are models for counterfactual outcomes that allow us to estimate causal effects from observational data. Unlike traditional regression models that condition on covariates, MSMs model the marginal distribution of potential outcomes:

$$\mathbb{E}[Y^a] = \beta_0 + \beta_1 a$$

where Y^a represents the potential outcome under treatment level a.

The key insight is that under the identifiability conditions (exchangeability, positivity, and consistency), IP weighting creates a pseudo-population where association is causation, allowing us to estimate MSM parameters using weighted regression.

Identifiability Conditions

For IP weighting to yield unbiased causal effects, three conditions must hold:

1. **Exchangeability**: $Y^a \perp A|L$ – no unmeasured confounding
2. **Positivity**: $0 < \Pr(A = a|L = l) < 1$ for all a, l – all treatment levels possible for all covariate patterns
3. **Consistency**: $Y = Y^a$ if $A = a$ – well-defined interventions

Intuitive Clarifications of Challenging Ideas

The Pseudo-Population Analogy

Imagine IP weighting creates a "parallel universe" where treatment assignment is random. In this universe, we can directly compare treated and untreated groups without worrying about confounding because the weighting process has broken the association between covariates and treatment.

Simple analogy: Think of IP weighting as creating a representative sample where each individual is replicated based on how unusual their treatment assignment was given their characteristics. Someone with covariates that strongly predict treatment but who didn't receive treatment gets more weight because they're "rare" and thus informative.

The Positivity Assumption in Practice

Positivity requires that all individuals have a non-zero probability of receiving each treatment level. Violations occur when:

1. **Structural violations**: Certain subgroups cannot receive specific treatments (e.g., only men receive prostate cancer treatments)
2. **Random violations**: Our sample lacks representation in certain strata

When positivity fails, IP weighting extrapolates based on model assumptions, which may be unreliable. Extreme weights (>10) often signal positivity issues.

! Stabilized vs. Non-Stabilized Weights

Non-stabilized weights: $W_i^A = \frac{1}{\Pr(A=a_i|L_i)}$

- Create a pseudo-population where everyone has equal probability of treatment
- Can produce extremely large weights and high variance

Stabilized weights: $SW_i^A = \frac{\Pr(A=a_i)}{\Pr(A=a_i|L_i)}$

- Create a pseudo-population that maintains the marginal treatment distribution
- Typically yield more efficient estimates
- Have mean approximately 1, making diagnostics easier

Simple, Hand-Calculation Numerical Example

Basic Example: Toy Dataset

Consider a simple dataset with 6 patients to understand the mechanics of IP weighting:

```
toy_data <- tibble(
    id = 1:10,
    age = c(30, 40, 50, 52, 53, 56, 60, 65, 70, 80), #
    Covariate
    treated = c(1, 1, 1, 0, 0, 1, 0, 1, 0, 0), #
    Treatment
```

```
    outcome = c(10, 12, 16, 13, 14, 8, 10, 11, 7, 6) # 
    Outcome
)
toy_data
```

Table 1: Toy dataset for IP weighting example

id	age	treated	outcome
1	30	1	10
2	40	1	12
3	50	1	16
4	52	0	13
5	53	0	14
6	56	1	8
7	60	0	10
8	65	1	11
9	70	0	7
10	80	0	6

Step 1: Estimate propensity scores manually

Since we have only one covariate (age), we can calculate propensity scores manually. Notice that younger patients are more likely to be treated:

Step 2: Calculate weighted means

```
weighted_mean_treated <- with(
    toy_data[toy_data$treated == 1, ],
    weighted.mean(outcome, w = ip_weight)
)
weighted_mean_untreated <- with(
    toy_data[toy_data$treated == 0, ],
    weighted.mean(outcome, w = ip_weight)
)
causal_effect <- weighted_mean_treated - 
weighted_mean_untreated
```

Table 2: Toy dataset with manually calculated propensity scores and IP weights

```r
toy_data <- toy_data |>
    mutate(
        # Simple deterministic propensity score based on
        age
        ps = ifelse(age < 55, 0.8, 0.2),
        # IP weights
        ip_weight = ifelse(treated == 1, 1 / ps, 1 / (1
        - ps))
    )
print(toy_data)
```

```
# A tibble: 10 x 6
      id   age treated outcome    ps ip_weight
   <int> <dbl>   <dbl>   <dbl> <dbl>     <dbl>
 1     1    30       1      10   0.8      1.25
 2     2    40       1      12   0.8      1.25
 3     3    50       1      16   0.8      1.25
 4     4    52       0      13   0.8      5
 5     5    53       0      14   0.8      5
 6     6    56       1       8   0.2      5
 7     7    60       0      10   0.2      1.25
 8     8    65       1      11   0.2      5
 9     9    70       0       7   0.2      1.25
10    10    80       0       6   0.2      1.25
```

```r
toy_data |>
    group_by(treated) |>
    summarize(
        unweighted_mean = mean(outcome),
        weighted_mean = weighted.mean(outcome, w =
        ip_weight),
        .groups = "drop"
    )
cat("Weighted mean (treated):",
round(weighted_mean_treated, 2), "\n")
```

```
Weighted mean (treated): 10.36
```

```r
cat("Weighted mean (untreated):",
round(weighted_mean_untreated, 2), "\n")
```

```
Weighted mean (untreated): 11.91
```

```r
cat("Estimated causal effect:", round(causal_effect, 2),
"\n")
```

```
Estimated causal effect: -1.55
```

Table 3: Toy dataset with manually calculated weighted means

treated	unweighted_mean	weighted_mean
0	10.0	11.90909
1	11.4	10.36364

Interpretation: The IP weighted analysis estimates that treatment increases the outcome by -1.55 units. Notice how the weights amplify the influence of the older treated patient (ID 8) and the younger untreated patient (ID 4), as these individuals have unusual treatment assignments given their age.

Advanced Example: Continuous Treatment

Now consider a scenario with continuous treatment (dose) and multiple covariates:

```
set.seed(123)
n <- 100
continuous_data <- tibble(
    id = 1:n,
    age = rnorm(n, 50, 10),
    severity = rnorm(n, 5, 2),
    dose = 0.1 * age + 0.3 * severity + rnorm(n, 0, 2),
    # Treatment dose
    outcome = 2 + 0.5 * dose + 0.2 * age + 0.4 *
    severity + rnorm(n, 0, 1)
)
```

For continuous treatments, we need to model the density of treatment rather than probability. This introduces additional complexity and sensitivity to model specification.

4 R Code Examples & Interpretation

Creating and Analyzing a DAG for Smoking Cessation

```
smoking_dag <- dagitty(
    'dag {
  bb="0,0,1,1"

  Age [pos="0.2,0.8"]
  Sex [pos="0.2,0.6"]
  Education [pos="0.2,0.4"]
  Baseline_Weight [pos="0.2,0.2"]
  Smoking_Intensity [pos="0.4,0.8"]
  Smoking_Duration [pos="0.4,0.6"]
  Physical_Activity [pos="0.4,0.4"]
```

```
  Exercise [pos="0.4,0.2"]

  Genetic_Predisposition [pos="0.6,0.7", exposure=true]
  Socioeconomic_Status [pos="0.6,0.3", exposure=true]

  Smoking_Cessation [pos="0.8,0.6"]
  Weight_Gain [pos="0.8,0.4"]

  Age -> Smoking_Cessation
  Age -> Weight_Gain
  Sex -> Smoking_Cessation
  Sex -> Weight_Gain
  Education -> Smoking_Cessation
  Education -> Weight_Gain
  Baseline_Weight -> Smoking_Cessation
  Baseline_Weight -> Weight_Gain
  Smoking_Intensity -> Smoking_Cessation
  Smoking_Intensity -> Weight_Gain
  Smoking_Duration -> Smoking_Cessation
  Smoking_Duration -> Weight_Gain
  Physical_Activity -> Smoking_Cessation
  Physical_Activity -> Weight_Gain
  Exercise -> Smoking_Cessation
  Exercise -> Weight_Gain

  Genetic_Predisposition -> Smoking_Cessation
  Genetic_Predisposition -> Weight_Gain
  Socioeconomic_Status -> Smoking_Cessation
  Socioeconomic_Status -> Weight_Gain

  Smoking_Cessation -> Weight_Gain
}'
)

# Convert to tidy format for plotting
tidy_smoking_dag <- tidy_dagitty(smoking_dag)
ggplot(tidy_smoking_dag, aes(x = x, y = y, xend = xend,
yend = yend)) +
```

```r
    geom_dag_point(
        aes(
            color = name %in%
                c("Genetic_Predisposition",
                "Socioeconomic_Status")
        ),
        show.legend = FALSE
    ) +
    geom_dag_edges() +
    geom_dag_text(aes(label = name), color = "black",
    size = 3) +
    theme_dag()
```

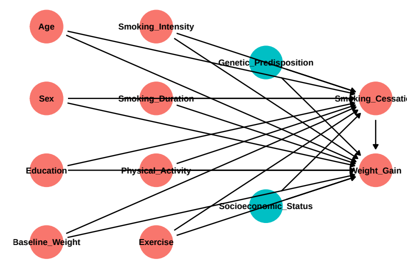


Figure 1: DAG for smoking cessation and weight gain example

This DAG illustrates the complex web of relationships in our smoking cessation example. The red nodes represent unmeasured confounders that would violate the exchangeability assumption if not addressed.

Comprehensive IP Weighting Analysis

```r
# Load and prepare NHEFS data
nhefs <- read_csv(here::here("data", "hernan2025",
"nhefs.csv")) |>
    mutate(
        qsmk = as.factor(qsmk),
        sex = as.factor(sex),
        race = as.factor(race),
        education = as.factor(education),
        exercise = as.factor(exercise),
        active = as.factor(active)
    )

# Estimate propensity scores with flexible model
ps_model <- glm(
    qsmk ~
```

```r
        sex +
        poly(age, 2) +
        race +
        education +
        poly(smokeintensity, 2) +
        poly(smokeyrs, 2) +
        exercise +
        active +
        poly(wt71, 2),
    family = binomial,
    data = nhefs
)

nhefs$ps <- predict(ps_model, type = "response")

# Calculate both stabilized and non-stabilized weights
nhefs <- nhefs |>
    mutate(
        # Marginal probability of treatment
        p_treat = mean(qsmk == 1),
        # Non-stabilized weights
        ipw_ns = ifelse(qsmk == 1, 1 / ps, 1 / (1 - ps)),
        # Stabilized weights
        ipw_s = ifelse(qsmk == 1, p_treat / ps, (1 -
        p_treat) / (1 - ps))
    )

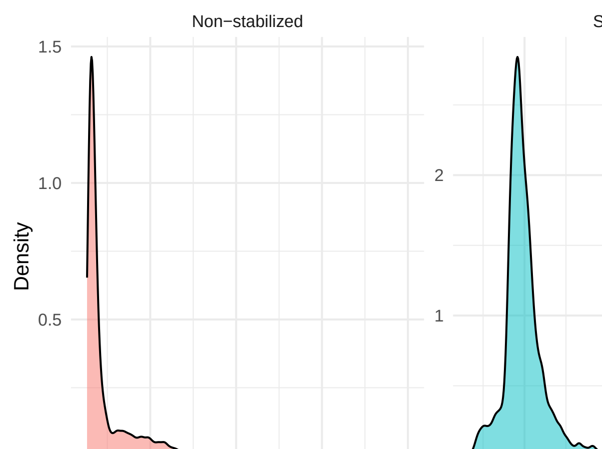
# Check weight distributions
weight_summary <- nhefs |>
    summarize(
        mean_ns = mean(ipw_ns),
        mean_s = mean(ipw_s),
        min_ns = min(ipw_ns),
        min_s = min(ipw_s),
        max_ns = max(ipw_ns),
        max_s = max(ipw_s),
        sd_ns = sd(ipw_ns),
```

```
        sd_s = sd(ipw_s)
    )
print(weight_summary)
```

```
# A tibble: 1 x 8
  mean_ns mean_s min_ns min_s max_ns max_s sd_ns  sd_s
    <dbl>  <dbl>  <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl>
1    2.00  0.999   1.06 0.331   16.0  4.21  1.43 0.284
```

```
# Visualize weight distributions
weight_plot <- nhefs |>
    select(ipw_ns, ipw_s) |>
    pivot_longer(
        everything(),
        names_to = "weight_type",
        values_to = "weight"
    ) |>
    mutate(
        weight_type = ifelse(
            weight_type == "ipw_ns",
            "Non-stabilized",
            "Stabilized"
        )
    ) |>
    ggplot(aes(x = weight, fill = weight_type)) +
    geom_density(alpha = 0.5) +
    facet_wrap(~weight_type, scales = "free") +
    labs(x = "Weight value", y = "Density", fill =
    "Weight type") +
    theme_minimal() +
    theme(legend.position = "none")
weight_plot
```

The weight distributions show that stabilized weights have much better properties - they are centered around 1 with less extreme values, which will lead to more efficient estimation.



```
# Assess covariate balance before and after weighting
covariates <- c("age", "wt71", "smokeintensity",
"smokeyrs")

# Pre-weight balance
balance_before <- nhefs |>
    group_by(qsmk) |>
    summarize(across(all_of(covariates), list(mean =
    mean, sd = sd))) |>
    pivot_longer(-qsmk, names_to = "variable", values_to
    = "value") |>
    separate(variable, into = c("variable", "stat"), sep
    = "_") |>
    pivot_wider(names_from = stat, values_from = value)

# Create weighted dataset
design_ns <- svydesign(ids = ~1, weights = ~ipw_ns, data
= nhefs)
design_s <- svydesign(ids = ~1, weights = ~ipw_s, data =
nhefs)

# Post-weight balance
balance_after_ns <- svyby(
    ~ age + wt71 + smokeintensity + smokeyrs,
    ~qsmk,
    design_ns,
    svymean
) |>
    pivot_longer(-qsmk, names_to = "variable", values_to
    = "mean") |>
    mutate(weight_type = "Non-stabilized")

balance_after_s <- svyby(
    ~ age + wt71 + smokeintensity + smokeyrs,
    ~qsmk,
    design_s,
    svymean
```

```
) ▷
    pivot_longer(-qsmk, names_to = "variable", values_to
    = "mean") ▷
    mutate(weight_type = "Stabilized")

# Combine results
balance_results <- bind_rows(
    balance_before ▷ mutate(weight_type =
"Unweighted"),
    balance_after_ns,
    balance_after_s
)

# Plot balance results
balance_plot <- ggplot(
    balance_results,
    aes(x = factor(qsmk), y = mean, color = weight_type,
    group = weight_type)
) +
    geom_point(position = position_dodge(width = 0.5)) +
    geom_line(position = position_dodge(width = 0.5)) +
    facet_wrap(~variable, scales = "free_y") +
    labs(x = "Smoking cessation", y = "Mean", color =
    "Weighting") +
    theme_minimal()
balance_plot
```

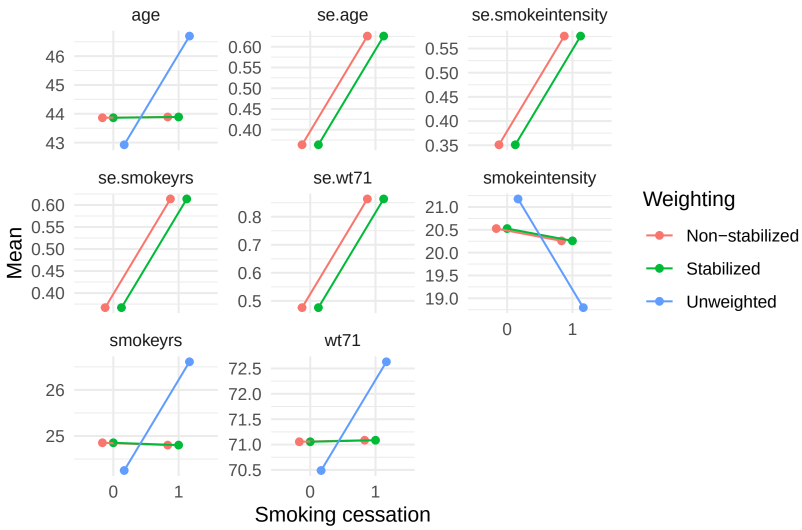


Figure 3: Balance assessment before and after IP weighting

The balance assessment shows how IP weighting improves comparability between treatment groups. After weighting, the means of covariates are much more similar between quitters and non-quitters.

```
# Fit MSM with both weight types
msm_ns <- svyglm(wt82_71 ~ qsmk, design = design_ns)
msm_s <- svyglm(wt82_71 ~ qsmk, design = design_s)

# Compare results
results_comparison <- tibble(
    Model = c("Non-stabilized", "Stabilized"),
    Estimate = c(coef(msm_ns)[2], coef(msm_s)[2]),
    SE = c(sqrt(diag(vcov(msm_ns)))[2],
    sqrt(diag(vcov(msm_s)))[2]),
    CI_lower = c(confint(msm_ns)[2, 1],
    confint(msm_s)[2, 1]),
    CI_upper = c(confint(msm_ns)[2, 2],
    confint(msm_s)[2, 2])
)

print(results_comparison)
```

```
# A tibble: 2 x 5
  Model          Estimate    SE CI_lower CI_upper
  <chr>             <dbl> <dbl>    <dbl>    <dbl>
1 Non-stabilized     3.52 0.520     2.50     4.54
2 Stabilized         3.52 0.520     2.50     4.54
```

Both weighting approaches yield similar point estimates (3.5 kg weight gain from smoking cessation), but the stabilized weights produce a slightly more precise estimate (smaller standard error).

5 Hands-On R Lab

Lab Objectives

1. Implement IP weighting for a causal question with real data
2. Diagnose balance, positivity, and model specification issues
3. Compare MSM results with traditional regression approaches
4. Explore effect modification using MSMs
5. Practice sensitivity analysis for unmeasured confounding

Pre-Lab Reading

Review Chapter 12 of Hernán & Robins (2020) and the positivity discussion in Neal (2020). Pay special attention to the concept of the pseudo-population and how weighting addresses confounding.

Data Preparation

```
```{r}
#| label: lab-data-prep
# Load and prepare NHEFS data for lab
nhefs_lab <-
read_csv("https://raw.githubusercontent.com/miguelherna
n/causal-inference-book/master/data/nhefs.csv") |>
mutate(
```
```

```
qsmk = as.factor(qsmk),
sex = as.factor(sex),
race = as.factor(race),
education = as.factor(education),
exercise = as.factor(exercise),
active = as.factor(active),
# Create a binary outcome for mortality analysis
death = as.factor(ifelse(is.na(wt82_71), 1, 0))
) |>
# Remove individuals with missing covariates for
simplicity
drop_na(age, wt71, smokeintensity, smokeyrs)
```
```

**Lab Tasks**

1. **Propensity Score Model Specification**

```
```{r}
#| label: task-1
# Student's task: Fit an appropriate propensity score
model
# Consider which covariates to include and functional
forms
# YOUR CODE HERE
ps_model <- glm(qsmk ~ ___ , family = binomial, data =
nhefs_lab)
```
```

2. **Weight Calculation and Diagnostics**

```
```{r}
#| label: task-2
# Student's task: Calculate weights and assess their
distribution
nhefs_lab$ps <- predict(ps_model, type = "response")
```

```
nhefs_lab <- nhefs_lab ▷
mutate(
ipw_ns = ifelse(___),  # Non-stabilized weights
ipw_s = ifelse(___)    # Stabilized weights
)

# Create visualizations of weight distributions
# YOUR CODE HERE
```
```

3. **Balance Assessment**

````
```{r}
#| label: task-3
# Student's task: Assess balance before and after weighting
# Use both statistical tests and visualizations
# YOUR CODE HERE
```
````

4. **MSM Estimation and Interpretation**

````
```{r}
#| label: task-4
# Student's task: Fit MSMs and interpret results
design_s <- svydesign(ids = ~1, weights = ~ipw_s, data = nhefs_lab)
msm <- svyglm(___)

# Interpret the coefficient for smoking cessation
# YOUR INTERPRETATION HERE
```
````

5. **Effect Modification Analysis**
````

````
```{r}
#| label: task-5
# Student's task: Test for effect modification by sex
# Fit an MSM with interaction term
# YOUR CODE HERE
```
````

Solutions

````
```{r}
#| label: solution-1
# Solution for task 1: Propensity score model
ps_model_solution <- glm(qsmk ~ sex + poly(age, 2) +
race + education +
           poly(smokeintensity, 2) + poly(smokeyrs, 2) +
             exercise + active + poly(wt71, 2),
           family = binomial, data = nhefs_lab)
summary(ps_model_solution)
```
````

````
```{r}
#| label: solution-2
# Solution for task 2: Weight calculation and
diagnostics
nhefs_lab$ps <- predict(ps_model_solution, type =
"response")
p_treat <- mean(nhefs_lab$qsmk == 1)

nhefs_lab <- nhefs_lab |>
mutate(
ipw_ns = ifelse(qsmk == 1, 1/ps, 1/(1-ps)),
ipw_s = ifelse(qsmk == 1, p_treat/ps,
(1-p_treat)/(1-ps))
)
````
````

````r
# Weight diagnostics
weight_summary <- nhefs_lab |>
summarize(
mean_ns = mean(ipw_ns),
mean_s = mean(ipw_s),
max_ns = max(ipw_ns),
max_s = max(ipw_s),
# Proportion of weights > 10
prop_large_ns = mean(ipw_ns > 10),
prop_large_s = mean(ipw_s > 10)
)
print(weight_summary)

# Visualize distributions
ggplot(nhefs_lab, aes(x = ipw_s)) +
geom_histogram(bins = 30) +
labs(title = "Distribution of stabilized weights", x =
"Weight", y = "Count")
```
````

Retrieval Quiz

1. What are the three key assumptions needed for IP weighting to yield unbiased causal effect estimates?
2. How do stabilized weights differ from non-stabilized weights, and why are they generally preferred?
3. What does a propensity score of 0.2 mean for a treated individual?
4. How can you diagnose positivity violations in your data?
5. Why is balance assessment crucial after IP weighting?

Reflection Prompts

1. How might violations of the positivity assumption affect your results in real-world applications, and what strategies could you use to address this?
````

2. What are the advantages of MSMs compared to traditional regression adjustment, and in what situations might traditional methods be preferred?
3. How would you explain the concept of IP weighting to a non-technical audience?

6 Common Pitfalls & Checks

! Weight Diagnostics and Truncation

Always check the distribution of IP weights. Extreme weights (typically >10) can indicate: - Positivity violations - Model misspecification - Rare treatment-covariate combinations
Strategies for handling extreme weights:

1. **Weight truncation**: Set upper and lower bounds for weights
2. **Stabilized weights**: Use marginal probabilities in numerator
3. **Alternative modeling**: Use machine learning methods for propensity scores
4. **Targeted selection**: Focus analysis on regions of common support

````
```{r}
#| label: weight-truncation
# Example of weight truncation
nhefs_lab <- nhefs_lab ▷
mutate(
ipw_s_truncated = case_when(
ipw_s > 10 ~ 10,
ipw_s < 0.1 ~ 0.1,
TRUE ~ ipw_s
)
)
```
````

> **Model Specification Challenges**
>
> The validity of IP weighting depends on correct specification of the propensity score model. Common issues:
>
> 1. **Omitted confounding**: Failure to include important confounders
> 2. **Non-linear relationships**: Assuming linearity when relationships are complex
> 3. **Interaction effects**: Missing important treatment-covariate interactions
> 4. **Overfitting**: Including too many covariates relative to sample size
>
> **Solutions**: - Use flexible functional forms (splines, polynomials) - Consider machine learning methods (random forests, BART) - Use variable selection carefully - Test different model specifications

Check Your Understanding

1. If the mean of your stabilized weights is 1.2, what might this indicate about your model?

- *Answer: This suggests possible model misspecification or positivity violations. The mean of stabilized weights should be approximately 1.*

2. How would you test for positivity violations in your data?

- *Answer: Examine the distribution of propensity scores, look for extreme weights, and check whether there are regions of covariate space where treatment assignment is deterministic.*

3. What's the difference between structural and random positivity violations?

- *Answer: Structural violations occur when certain subgroups cannot receive specific treatments (e.g., only women receive hysterectomy). Random violations occur when our sample lacks representation in certain strata due to sampling variability.*

7 Advanced Teaching Activities & Interactivity

Think-Pair-Share Exercise

"Discuss with a partner: In what situations might IP weighting be preferred over propensity score matching? What are the advantages and disadvantages of each approach? Consider aspects like efficiency, robustness to model misspecification, and handling of continuous treatments."

Flipped Classroom Preparation

Before the next class, explore the impact of different weight truncation strategies on effect estimates using the NHEFS data. Prepare to present: 1. How different truncation thresholds affect point estimates 2. The trade-off between bias and efficiency in truncation decisions 3. Recommendations for practice based on your exploration

Interactive Sensitivity Analysis

````
```{r}
#| label: sensitivity-analysis
#| eval: false
# Template for sensitivity analysis for unmeasured
confounding
sensitivity_analysis <- function(data, outcome,
treatment, confounder_strength = 1) {
# This is a simplified example - real sensitivity
analysis would be more complex
results <- list()

for (strength in seq(0.5, 2, by = 0.1)) {
# Modify weights based on assumed confounder strength
modified_weights <- data$ipw_s * strength

# Re-estimate effect with modified weights
````
````

```
design_modified <- svydesign(ids = ~1, weights =
~modified_weights, data = data)
model_modified <- svyglm(reformulate(treatment,
outcome), design = design_modified)

results[[as.character(strength)]] <-
tidy(model_modified) ▷
filter(term == treatment) ▷
mutate(strength = strength)
}

bind_rows(results)
}

# Students can explore how results change with different
assumptions
# about unmeasured confounding strength
```

8 Appendix: Advanced Theory & Derivations

Efficient Influence Functions and Double Robustness

The efficient influence function for the IP weighted mean is:

$$\psi(O) = \frac{I(A=a)Y}{\Pr(A=a|L)} - E[Y^a] + \left(\frac{I(A=a)}{\Pr(A=a|L)} - 1\right) E[Y|A=a,L]$$

This representation shows the double robustness property of IP weighting estimators. The estimator is consistent if either: 1. The propensity score model $\Pr(A=a|L)$ is correctly specified, OR 2. The outcome model $E[Y|A=a,L]$ is correctly specified

This double robustness property provides protection against model misspecification.

Large Sample Properties

Under regularity conditions, IP weighted estimators are consistent and asymptotically normal:

$$\sqrt{n}(\hat{\beta} - \beta_0) \rightsquigarrow N(0, E[\psi\psi^T])$$

where ψ is the efficient influence function. The asymptotic variance can be estimated using the sandwich estimator:

$$\hat{Var}(\hat{\beta}) = \left(\sum_{i=1}^{n} W_i X_i X_i^T\right)^{-1} \left(\sum_{i=1}^{n} W_i^2 \hat{\epsilon}_i^2 X_i X_i^T\right) \left(\sum_{i=1}^{n} W_i X_i X_i^T\right)^{-1}$$

where W_i are the weights, X_i are the covariates, and $\hat{\epsilon}_i$ are the residuals.

Comparison with Other Methods

IP Weighting vs. Regression Adjustment:

- IP weighting models the treatment process, regression models the outcome process
- IP weighting is more transparent about its assumptions
- Regression can be more efficient when models are correctly specified
- IP weighting naturally handles time-varying treatments

IP Weighting vs. Matching:

- Both create comparable groups
- Matching discards data (only uses matched observations)
- IP weighting uses all data but can be inefficient with extreme weights
- Matching is more intuitive for many researchers

IP Weighting vs. G-computation:

- IP weighting is semi-parametric, G-computation is parametric

- G-computation requires modeling the outcome process
- IP weighting requires modeling the treatment process
- Both can be combined in doubly robust estimators

References

Cunningham, S. (2021). *Causal Inference: The Mixtape*. Yale University Press. https://doi.org/10.2307/j.ctv1c29t27

Hernán, M. Á., & Robins, J. M. (2020). *Causal inference: What if*. Chapman & Hall/CRC.

Imbens, G., & Rubin, D. B. (2015). *Causal inference for statistics, social, and biomedical sciences: An introduction*. Cambridge University Press.

Neal, B. (2020). *Introduction to Causal Inference from a Machine Learning Perspective*.

Pearl, J. (2009). *Causality: Models, reasoning, and inference* (2nd ed.). Cambridge University Press.

VanderWeele, T. J. (2015). *Explanation in causal inference: Methods for mediation and interaction*. Oxford University Press.